TensorFlow vs. PyTorch: Inference Performance on the Jetson Orin Nano

Edge devices have limited compute, so inference speed matters. TensorFlow and PyTorch can run the same trained model, but they may perform differently on embedded hardware. The question we are seeking to ask is: **how do TensorFlow and PyTorch compare in inference latency and throughput when running equivalent models on the NVIDIA Jetson Orin Nano?**

We test three workloads: K-nearest neighbors (KNN), a linear support vector machine (SVM), and a YOLO image classifier. Each runs in both frameworks, so there are six cases.

Model	PyTorch	TensorFlow
KNN	Distance, nearest-neighbor selection, and voting in PyTorch ops	Same calculations in TensorFlow ops
Linear SVM	Scores from transferred weights and bias	Same scoring with identical weights and bias
YOLO classifier	Original PyTorch model	Validated TensorFlow export

We compare frameworks within each model only. KNN and YOLO take different inputs and do different work, so we won't rank them against each other. Instead, we look at whether the framework gap changes between distance calculations, linear classification, and neural-network inference.

Data and Model Setup

We will use a small-labeled image set, `[DATASET, e.g. 10 ImageNet classes, resized to 224x224]`, with one fixed train/test split. KNN and SVM get the same numerical features (such as HOG) with the same normalization. YOLO gets raw images preprocessed to its input requirements.

The SVM is trained once in scikit-learn, and its weights and bias are loaded into both frameworks, where tensor ops compute the scores. Both KNN versions use the same stored examples, labels, distance metric, k, and tie-breaking rule. scikit-learn is only used for training and as a reference check. It never runs the predictions we time.

For YOLO, we use the classification variant with pretrained weights and export it to TensorFlow. This is our riskiest step, so we test it first. If the export can't be validated, we substitute a smaller classifier such as MobileNetV2.

Before any timing, we check that predicted labels match and that numerical outputs agree within a set floating-point tolerance. Test accuracy confirms nothing broke during porting or conversion.

Benchmarking

Everything runs on the same Jetson Orin Nano. Held constant across frameworks:

- Same weights, inputs, and preprocessing
- FP32 precision
- Batch size 1 for latency tests
- Same execution device for each model (we will state CPU or GPU per model)
- Same power mode, cooling, and no background workloads

After warm-up, we run repeated latency measurements and a sustained throughput test. GPU timing waits for the work to finish, so we measure completed computation and not just submission.

Measurement	Why
Median latency	Typical response time
95th-percentile latency	Slow cases and variability
Throughput (FPS)	Images processed per second
Agreement and accuracy	Confirms both frameworks compute the same thing
Model-only vs. full pipeline time	Separates inference from preprocessing and feature extraction

Full pipeline time covers everything from input image to prediction, so a fast classifier can't hide an expensive feature step. If setup goes smoothly, we will also record peak memory and power with `tegrastats` as an optional extra.

Timeline

Phase	Main activities
Setup and preparation	Verify framework compatibility, select data, and establish model parameters
Implementation and validation	Implement KNN and SVM, export YOLO, and verify equivalent predictions
Benchmarking	Collect controlled latency, throughput, and accuracy measurements
Analysis and delivery	Explain results, create figures, and document reproducible experiments

Deliverables

A GitHub repo with validated implementations, benchmark scripts, comparison plots, and a written analysis. Our results will show which framework is faster for each of the three workloads on the Jetson Orin Nano, by how much, and whether preprocessing limits the gain. Conclusions apply to our implementations and test conditions on this board.